NSP, Combined Loss & Parameter Count.

BERT's second pretraining head is a single sigmoid on the [CLS] vector. This topic derives next-sentence prediction, combines it with the MLM loss, then audits BERT-base parameter by parameter to the famous ≈110M figure — every line computed exactly.

THE EQUATIONS

$$p = \sigma(w \cdot h_{\texttt{[CLS]}} + b), \quad \mathcal{L}_{\text{NSP}} = -[y \log p + (1-y)\log(1-p)], \quad \mathcal{L} = \mathcal{L}_{\text{MLM}} + \mathcal{L}_{\text{NSP}}$$

Worked example. A concrete positive pair and a concrete negative pair: logit → probability → loss. Then a full BERT-base parameter-budget table summing to ≈110M.

Topic 3 of 3 in this module · companion to the course page · v1.0

Contents

1 Next-sentence prediction (NSP)

Alongside masked-language modelling, the original BERT trains on a sentence-pair task. Two segments A and B are concatenated as `[CLS] A [SEP] B [SEP]`; half the time B is the *actual* next sentence ($y = 1$), half the time B is a random sentence from the corpus ($y = 0$). The pooled `[CLS]` hidden vector $h_{\texttt{[CLS]}} \in \mathbb{R}^d$ is fed to a single logistic unit:

$$\boxed{p = \sigma(z), \quad z = w \cdot h_{\texttt{[CLS]}} + b, \quad \sigma(z) = \frac{1}{1 + e^{-z}}} \tag{1}$$

with the binary cross-entropy loss

$$\boxed{\mathcal{L}_{\text{NSP}} = -\left[y \log p + (1 - y) \log(1 - p)\right]} \tag{2}$$

where p is the predicted probability that B follows A.

> NSP is a yes/no question asked of one vector. The `[CLS]` token attends over the whole pair, so its final representation must summarise "do these two segments belong together?" into a single scalar logit. It is the simplest possible classification head: a dot product plus a bias.

2 The combined pretraining loss

BERT optimises both objectives at once. For a batch, the total loss is the sum of the per-position MLM term (Topic 1) and the per-pair NSP term:

$$\boxed{\mathcal{L} = \mathcal{L}_{\text{MLM}} + \mathcal{L}_{\text{NSP}} = -\sum_{i \in \mathcal{M}} \log \hat{p}_{i,x_i} \; - \; [y \log p + (1-y)\log(1-p)]} \tag{3}$$

Both heads share the *same* encoder; gradients from (3) flow back through both the MLM output projection and the NSP weight w into the common transformer stack. There is no weighting coefficient in the original formulation — the two losses are simply added.

3 Worked NSP example

Take $d = 4$, NSP weight $w = (0.4, -0.3, 0.8, 0.1)$ and bias $b = 0.2$.

3.1 Positive pair ($y = 1$)

Let $h_{\texttt{[CLS]}} = (1.0,\ 0.5,\ -0.2,\ 2.0)$. Then

$$z = w \cdot h + b = 0.4(1.0) - 0.3(0.5) + 0.8(-0.2) + 0.1(2.0) + 0.2 = 0.49,$$

$$p = \sigma(0.49) = \frac{1}{1 + e^{-0.49}} = 0.620106, \qquad \mathcal{L}_{\text{NSP}} = -\log(0.620106) = 0.477864.$$

The model leans "next sentence" ($p > 0.5$) and is correct, so the loss is modest.

3.2 Negative pair ($y = 0$)

Let $h_{\texttt{[CLS]}} = (-0.5,\ 1.2,\ 0.3,\ -1.0)$. Then

$$z = 0.4(-0.5) - 0.3(1.2) + 0.8(0.3) + 0.1(-1.0) + 0.2 = -0.22,$$

$$p = \sigma(-0.22) = 0.445221, \qquad \mathcal{L}_{\text{NSP}} = -\log(1-p) = -\log(0.554779) = 0.589185.$$

Here $p < 0.5$, correctly predicting "not next", but with less confidence, so the loss is larger.

Predicted $p = \sigma(z)$ for the two pairs (intensity scaled to the max)

	positive ($y=1$)	negative ($y=0$)
p(is-next)	**0.6201**	0.4452
$\mathcal{L}_{\text{NSP}}$	0.4779	0.5892

Mean NSP loss over the two examples: $\frac{1}{2}(0.477864 + 0.589185) = 0.533525$.

For binary cross-entropy with a sigmoid, the gradient on the logit collapses to the same "predicted minus target" residual as softmax: $\partial \mathcal{L}_{\text{NSP}} / \partial z = p - y$. For the positive pair that is $0.620106 - 1 = -0.379894$ (push z up); for the negative pair $0.445221 - 0 = +0.445221$

(push z down).

4 BERT-base parameter count

We now audit every parameter of BERT-base: $V = 30522$, hidden $H = 768$, max positions $P = 512$, segment types $S = 2$, layers $L = 12$, FFN inner $4H = 3072$. All numbers below are exact integer counts.

4.1 Embeddings

Three lookup tables plus the embedding LayerNorm (γ, β, each of size H):

$$\underbrace{30522 \cdot 768}_{\text{token}} + \underbrace{512 \cdot 768}_{\text{position}} + \underbrace{2 \cdot 768}_{\text{segment}} + \underbrace{2 \cdot 768}_{\text{LN}} = 23{,}440{,}896 + 393{,}216 + 1{,}536 + 1{,}536 = 23{,}837{,}184.$$

The token table alone is 23.44M — over a fifth of the whole model.

4.2 One encoder layer

Multi-head attention has four $H \times H$ projections (Q, K, V, output), each with an H bias:

$$\text{MHA} = 4(H^2 + H) = 4(589{,}824 + 768) = 2{,}362{,}368.$$

The position-wise FFN is two affine maps $H \to 4H \to H$:

$$\text{FFN} = (H \cdot 4H + 4H) + (4H \cdot H + H) = (2{,}359{,}296 + 3{,}072) + (2{,}359{,}296 + 768) = 4{,}722{,}432.$$

Two LayerNorms per layer (post-attention, post-FFN), each $2H$:

$$\text{LN} = 2 \cdot 2H = 3{,}072.$$

Per layer: $2{,}362{,}368 + 4{,}722{,}432 + 3{,}072 = 7{,}087{,}872 \approx 7.09\text{M}$.

4.3 Stack, pooler, total

$$\text{encoder} = 12 \times 7{,}087{,}872 = 85{,}054{,}464, \qquad \text{pooler} = H^2 + H = 590{,}592.$$

$$\boxed{23{,}837{,}184 + 85{,}054{,}464 + 590{,}592 = 109{,}482{,}240 \approx 110\text{M}} \tag{4}$$

4.4 Parameter-budget table

BERT-base parameter budget (intensity scaled to the largest block)

Component	Formula	Params	Share
Token embeddings	$30522 \cdot 768$	23,440,896	21.4%
Position embeddings	$512 \cdot 768$	393,216	0.4%
Segment embeddings	$2 \cdot 768$	1,536	0.0%
Embedding LayerNorm	$2 \cdot 768$	1,536	0.0%
MHA / layer	$4(768^2+768)$	2,362,368	2.2%
FFN / layer	$2 \cdot 768 \cdot 3072+\ldots$	4,722,432	4.3%
LayerNorms / layer	$2 \cdot 2 \cdot 768$	3,072	0.0%
Encoder ($\times 12$)	$12 \cdot 7{,}087{,}872$	85,054,464	**77.7%**
Pooler	768^2+768	590,592	0.5%
Total		**109,482,240**	**100%**

The 109.48M backbone is what is conventionally quoted as "BERT-base, 110M". The two pretraining heads add a little more: the MLM transform (H^2+H = 590,592), its LayerNorm ($2H$ = 1,536) and output bias (V = 30,522), plus the NSP head ($2H+2$ = 1,538) — about 0.62M, for $\approx$ 110.11M including heads. The decoder *weight* is tied to the token embedding, so it adds no new parameters.

5 Properties / consequences that matter

1. **The encoder dominates**: 77.7% of parameters live in the 12 transformer layers; embeddings are 21.8% and almost all of that is the token table.
2. **FFN > attention**: within a layer the FFN holds 4.72M vs MHA's 2.36M — two-to-one — because of the 4× inner expansion.
3. **NSP is essentially free**: $w \in \mathbb{R}^{768}$ plus a bias is 769 parameters, a rounding error against 110M, yet later work (RoBERTa) found dropping NSP entirely *helps*.
4. **Combined loss, shared body**: the same 85M-parameter encoder serves both heads; this parameter sharing is what makes the pretrained representation transfer.

Do not forget the biases when counting. The attention projections contribute $4H$ bias parameters and the FFN $4H+H$ more per layer; omitting them undercounts each layer by $\sim$6,912 parameters ($\sim$83k across 12 layers). And the FFN inner width is $4H = 3072$, *not* H — mistaking it for H halves the layer estimate.

6 Where this sits in the track

This closes Module 1.1. Topic 1 gave the MLM loss and its softmax-residual gradient; Topic 2 explained the 80/10/10 inputs that feed it; here NSP and the combined loss complete BERT's

pretraining objective, and the parameter audit grounds "110M" in arithmetic you can reproduce line by line. The next module builds on this pretrained encoder for fine-tuning.

A Reproducible (NumPy)

NSP inputs: $w = (0.4, -0.3, 0.8, 0.1)$, $b = 0.2$; positive $h = (1, 0.5, -0.2, 2)$ gives $z = 0.49$, $p = 0.620106$, $\mathcal{L} = 0.477864$; negative $h = (-0.5, 1.2, 0.3, -1)$ gives $z = -0.22$, $p = 0.445221$, $\mathcal{L} = 0.589185$. Param totals: embeddings 23,837,184; per layer 7,087,872; encoder 85,054,464; pooler 590,592; total 109,482,240.

```
import numpy as np
sig = lambda z: 1/(1+np.exp(-z))
w, b = np.array([0.4,-0.3,0.8,0.1]), 0.2
for h, y in [(np.array([1.,.5,-.2,2.]),1), (np.array([-.5,1.2,.3,-1.]),0)]:
    z = w @ h + b; p = sig(z)
    L = -(y*np.log(p) + (1-y)*np.log(1-p))
    print(round(z,4), round(p,6), round(L,6))   # 0.49 .620106 .477864 ; -0.22 .445221 .5891

V,H,P,S,Ln,F = 30522,768,512,2,12,3072
emb = V*H + P*H + S*H + 2*H                      # 23,837,184
mha = 4*(H*H + H)                                # 2,362,368
ffn = (H*F + F) + (F*H + H)                       # 4,722,432
ln  = 2*(2*H)                                    # 3,072
layer = mha + ffn + ln                           # 7,087,872
total = emb + Ln*layer + (H*H + H)               # 109,482,240  (~110M)
print(emb, layer, Ln*layer, total)
```